Nitya CloudTech Pvt Ltd.

# SQL Scenario-Based Interview Questions & Answers



## 1. Retrieve Data with Conditions

**Q:** How would you find employees with salaries greater than the average salary in the employees table?

**A:**

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

## 2. Using JOINS

**Q:** Retrieve a list of employees who have made sales. Include their names, employee ID, and sale amount from `employees` and `sales` tables.

**A:**

```
SELECT e.employee_id, e.name, s.sale_amount
FROM employees e
INNER JOIN sales s ON e.employee_id = s.employee_id;
```

## 3. Aggregate Functions

**Q:** Write a query to find the maximum and minimum salary in each department from the `employees` table.

**A:**

```
SELECT department_id, MAX(salary) AS max_salary, MIN(salary) AS
min_salary
FROM employees
GROUP BY department_id;
```

## 4. Handling NULL Values

**Q:** How would you count the number of orders where the shipping date is NULL in the `orders` table?

**A:**

```
SELECT COUNT(*) AS orders_with_no_shipping_date
FROM orders
WHERE shipping_date IS NULL;
```

## 5. Using CASE Statements

**Q:** Create a query to categorize employees into 'High', 'Medium', and 'Low' salary ranges in the `employees` table.

**A:**

```
SELECT employee_id, name,
       CASE
           WHEN salary > 100000 THEN 'High'
           WHEN salary BETWEEN 50000 AND 100000 THEN 'Medium'
           ELSE 'Low'
       END AS salary_category
FROM employees;
```

## 6. Date Filtering

**Q:** Write a query to fetch orders placed in the last 30 days from the `orders` table.

**A:**

```
SELECT *
FROM orders
WHERE order_date >= DATEADD(day, -30, GETDATE());
```

## 7. Subqueries

**Q:** Retrieve all customers who have made more than 5 orders from the `customers` and `orders` tables.

**A:**

```
SELECT customer_id, name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    HAVING COUNT(order_id) > 5
);
```

## 8. Using DISTINCT

**Q:** How would you find unique product names from the `products` table?

**A:**

```
SELECT DISTINCT product_name
FROM products;
```

## 9. Updating Records

**Q:** Update the `salary` of all employees in the 'Sales' department by 10% in the `employees` table.

**A:**

```
UPDATE employees
SET salary = salary * 1.1
WHERE department = 'Sales';
```

## 10. DELETE Operation

**Q:** Delete records of employees who have not made any sales in the past year.

**A:**

```
DELETE FROM employees
WHERE employee_id NOT IN (
    SELECT employee_id
    FROM sales
    WHERE sale_date >= DATEADD(year, -1, GETDATE())
);
```

## 11. Using GROUP BY with HAVING

**Q:** Write a query to find departments where the average salary is greater than $60,000.

**A:**

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

## 12. Self JOIN

**Q:** Find pairs of employees who have the same salary.

**A:**

```
SELECT e1.employee_id, e2.employee_id, e1.salary
FROM employees e1
JOIN employees e2 ON e1.salary = e2.salary AND e1.employee_id !=
e2.employee_id;
```

## 13. Window Functions

**Q:** Write a query to find each employee's rank based on salary within their department.

**A:**

```
SELECT employee_id, department_id, salary,
       RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)
AS salary_rank
FROM employees;
```

## 14. Row Numbering

**Q:** Select the top 3 highest-paid employees in each department.

**A:**

```
SELECT employee_id, department_id, salary
FROM (
    SELECT employee_id, department_id, salary,
           ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) AS row_num
    FROM employees
) AS ranked
WHERE row_num <= 3;
```

## 15. Pivot Table

**Q:** Write a query to show total sales amount by month in columns for a specific year.

**A:**

```
 customer_id,
       SUM(CASE WHEN MONTH(order_date) = 1 THEN amount ELSE 0 END) AS
Jan,
       SUM(CASE WHEN MONTH(order_date) = 2 THEN amount ELSE 0 END) AS
Feb,
       ...
       SUM(CASE WHEN MONTH(order_date) = 12 THEN amount ELSE 0 END)
AS Dec
FROM sales
WHERE YEAR(order_date) = 2024
GROUP BY customer_id;
```

## 16. Data Formatting

**Q:** Format a date field in the `orders` table as 'YYYY-MM-DD'.

**A:**

```
SELECT order_id, FORMAT(order_date, 'yyyy-MM-dd') AS formatted_date
FROM orders;
```

## 17. Finding Duplicates

**Q:** Identify duplicate entries in the `products` table based on product name and price.

**A:**

```
SELECT product_name, price, COUNT(*)
FROM products
GROUP BY product_name, price
HAVING COUNT(*) > 1;
```

## 18. Using UNION

**Q:** Combine results from the `products` table and the `inventory` table to show a complete list of products.

**A:**

```
SELECT product_id, product_name FROM products
UNION
SELECT item_id, item_name FROM inventory;
```

## 19. Recursive Query (CTE)

**Q:** Create a query to calculate a hierarchy level for employees based on a `manager_id` field.

**A:**

```
WITH EmployeeHierarchy AS (
    SELECT employee_id, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.manager_id, eh.level + 1
    FROM employees e
    JOIN EmployeeHierarchy eh ON e.manager_id = eh.employee_id
)
SELECT * FROM EmployeeHierarchy;
```

## 20. Finding Gaps in Sequences

**Q:** Identify gaps in a sequential list of invoice numbers.

**A:**

```
SELECT (invoice_number + 1) AS start_gap,
       (LEAD(invoice_number, 1) OVER (ORDER BY invoice_number) - 1)
AS end_gap
```

```
FROM invoices
WHERE (invoice_number + 1) <> LEAD(invoice_number, 1) OVER (ORDER BY
invoice_number);
```